

Data Preprocessing, Feature Engineering and Augmentation

Text Data

Sequence Data

- Sequence data is a type of data where the order of data points matters.
- Each element in the sequence depends on the elements that come before (and sometimes after) it.
- Sequence data can be time based or structure / event based.
- Sequence data can have different lengths for different examples.
- Traditional feed-forward neural networks do not handle sequence order naturally.
- Specialized models, such as Recurrent Neural Networks (RNNs), are designed to work with sequence data.

1. Time Series Data

- In time series data, observations are ordered by time.
- Each data point is associated with a specific time, and the sequence follows this time order.

2. Non-Time Based Sequence Data

- In this type of sequence data, the order is important, but it is not based on time.
- Instead, the order comes from structure, rules, or events.
- Includes:
 - Text data, where words appear in a meaningful order
 - Audio data, such as speech signals represented as a sequence of frames

Text Representation

- Text is one of the most common forms of sequence data.
- It can be viewed as a sequence of characters or a sequence of words, though in practice we often work at the level of words or sub-words.
- Machine learning and Deep learning models cannot process raw text directly.
- They operate only on numeric data, usually in the form of tensors.
- Therefore, the first challenge in building language models is to convert text into numbers.
- This process is called text encoding or text representation, and it involves transforming text into numeric vectors or tensors.
- The text-encoding process typically consists of two main steps:
 1. **Tokenization**: The text is broken down into smaller units called tokens (such as words, sub-words, or characters).
 2. **Vectorization**: Tokens of entire documents are mapped to numeric vectors.
- Hence, the overall text preprocessing pipeline includes text preparation, tokenization, text data augmentation (applied to the training data), vocabulary building, and vectorization. Padding and truncation are also required by certain algorithms.

Text Preparation

- Some steps that are applied to raw text before tokenization to ensure consistent token boundaries are:
 - Lowercasing
 - Removing or normalizing punctuation
 - Removing HTML tags, URLs, emojis
 - Normalizing whitespace (collapsing multiple spaces, tabs and newlines into one, trimming leading / trailing spaces)
 - Expanding contractions

Contractions are shortened forms of words like don't for do not

Tokenization

- Tokenization can be done in multiple ways:
 - Segment text into characters, and transform each character into a vector; here each character is a token.
 - Segment text into words, and transform each word into a vector; here each word is a token.
 - Extract n-grams of words or characters, and transform each n-gram into a vector; here each n-gram is a token.
N-grams are overlapping groups of multiple consecutive words or characters.
These are popularly used in shallow algorithms, not preferred in deep learning models like RNNs.
- In character level tokenization, we do not need to worry about the unknown tokens.
- Also, the generated sequences are very long, taking up more computational power.
- These models are not very good in predicting long range dependencies.
- However there could be some applications that require character level modeling and the area is currently being explored.

Corpus and Vocabulary (Dictionary of Words)

- A corpus is the complete collection of text data used in a particular application or task. It may consist of documents, sentences, paragraphs, or any other text sources.
- From the corpus, we build a vocabulary, also called a dictionary.
The vocabulary is the set of unique words or tokens that appear in the corpus and are selected for modeling.
It is used to assign a unique numeric index to each word or token so that text can be represented numerically.

Building a Vocabulary

- One common way to build a vocabulary is to select the top 10,000 most frequently occurring words or tokens from the corpus.
- Each word in the vocabulary is mapped to a unique index.
- Depending on the task, additional *special tokens* may be included in the vocabulary:
<UNK> – represents unknown or rare words not present in the vocabulary
<EOS> – marks the end of a sentence
Punctuation tokens – explicitly represent punctuation marks
- Each special token is also assigned its own numeric index.
- The size of the vocabulary depends on the application and the corpus. Typical commercial applications: 30,000 – 50,000 tokens. Larger systems: up to 100,000 tokens. Large internet-scale applications may use vocabularies with more than 1 million tokens
- A larger vocabulary captures more information but increases memory and computational requirements.

Word	Index
a	1
and	2
apple	3
...	...
hat	375
...	...
orange	2786
...	...
zoo	9,999
<UNK>	10,000

Vectorization

- Vectorization is the process of converting text units (tokens or entire documents) into numeric vectors so that they can be used as input to a machine learning model.
- Depending on the representation method, vectorization may produce:
 - Document level vectorization
 - Token level vectorization

Document-level vectorization

- Produces a single vector per document.
- These are frequency-based text representations which convert an entire document into a numeric vector by counting the occurrence of words or tokens from a fixed vocabulary.
- These methods treat a document as a collection of words and ignore word order and grammar, hence cannot capture semantics or context.
- Used with traditional/shallow ML algorithms like Naive Bayes, SVM, Logistic Regression.
- Common techniques include bag of words (BoW) and term frequency - inverse document frequency (TF-IDF)

Bag of Words (BoW)

- The Bag of Words model represents a document as a vector of word counts.
- The vectors produced are high dimensional and sparse.

Term Frequency–Inverse Document Frequency (TF-IDF)

- Like Bag of Words, TF-IDF also produces one vector per document and relies on a vocabulary constructed from the training corpus.
- TF-IDF improves upon raw word counts by assigning lower weights to common words and higher weights to informative words.
- This helps reduce the influence of stopwords and highlight discriminative words.

Token-level vectorization

- Produces a sequence of vectors, one per token, which are grouped into sequence tensors and provided as input to deep neural networks.
- Each token corresponds to one position in the input sequence, and its vector values serve as the input features.
- Used with deep learning models like recurrent neural networks (RNNs) and Transformers
- Common techniques include one-hot encoding and word embeddings.

One-Hot Vector Representation of Words

- If a document has a vocabulary V with $|V|$ words, each word can be represented with a one-hot vector.
- A one-hot vector is a $|V|$ -dimensional representation vector, where exactly one element is 1 and all other elements are 0.
- Each word is assigned a unique index. To represent a unique word w_i , set the i^{th} component of the vector to be 1, and zero out all of the other components.
- One-hot encoding has several limitations:
 1. The vectors obtained are sparse (mostly made of zeros), and very high-dimensional (same dimensionality as the number of words in the vocabulary).
 2. The scheme does not capture any semantic or contextual correlations between the words.
- The encodings seem rather arbitrary; does not make similar words into similar vectors.
- **Examples**
The words “jump” and “leap” have similar meanings but very different one-hot vectors. The words “man” and “woman” are contextually related but have unrelated vectors. Similarly, the words “apple” and “orange” are both fruits, yet their vectors are orthogonal. As a result, a model cannot easily generalize from “I want a glass of apple juice” to “I want a glass of orange juice”.

Word Embeddings

- Word embeddings are dense, low-dimensional floating-point vectors, hence are more memory efficient as compared to one-hot vectors.
- They capture inherent meaning or semantics of a word.
- Unlike one-hot vectors:
 - Word embeddings are learned from data.
 - Similar words are placed close to each other in the embedding space.
- It is common to see word embeddings that are 256-dimensional, 512-dimensional, or 1,024-dimensional when dealing with very large vocabularies; on the other hand, one-hot encoding words generally leads to vectors that are 20,000-dimensional or greater; hence word embeddings pack more information into far fewer dimensions.
- What makes a good word-embedding space depends heavily on the task.
 - If the corpus is large, embeddings can be learned directly from the dataset.
 - For small datasets, pre-trained embeddings can be used with little or no fine-tuning.
- Some popular word embedding methods are:
 - Word2Vec
 - GloVe (Global Vectors)
 - FastText (subword embeddings)
 - Contextual embeddings such as BERT; here the same word can have different vectors depending on the sentence.

Sequence Padding and Truncation

- After tokenization, text sequences typically have variable lengths, as different sentences contain different numbers of tokens.
- Many neural network models process data in batches and therefore require inputs to have a fixed sequence length.
- Padding and truncation are applied as part of the vectorization and input preparation stage, after tokens have been converted into numerical IDs.
- Padding adds special padding tokens (e.g., <PAD> or 0) to shorter sequences so that they reach a predefined maximum length, while truncation removes tokens from longer sequences that exceed this length. Together, these steps ensure uniform input dimensions, control memory usage, and enable efficient batch processing.
- Padding tokens do not carry semantic meaning and are typically ignored by the model through masking mechanisms.

Text Data augmentation

- Text data augmentation is the process of artificially increasing the size and diversity of a text dataset by generating new text samples from existing ones while approximately preserving the original label.
- Because text is discrete and meaning-sensitive, text augmentation techniques must balance noise injection and semantic preservation.
- Text augmentation techniques can be grouped into two major categories, based on whether they aim to preserve meaning explicitly.

Noise-Based (Non-Semantic, Rule-Based) Techniques

- These techniques do not understand language semantics.
- They introduce controlled randomness to improve model robustness, hence regularizing the model.
- Some common techniques are:
 - **Random Deletion:** Randomly removes words with a fixed probability.
 - **Random Insertion:** Inserts a word at a random position; it could be truly random word insertion (rare), or synonym-based insertion (more common).
 - **Random Swap:** Randomly swaps the positions of two words.
 - **Character-Level Noise:** Introduces spelling or character-level errors.

Semantic (Meaning-Preserving) Techniques

- These techniques aim to preserve the meaning of the text, either approximately or strongly.
- These methods use pretrained language models that capture context, grammar, and semantics.
- Strong meaning-preserving text augmentation generally requires deep learning models, though not all of them are fully generative.
- Some common techniques are:
 - **Dictionary-Based Synonym Replacement:** Replaces words using synonym dictionaries (e.g., WordNet).
 - **Contextual Word Replacement:** Uses masked language models (e.g., BERT, RoBERTa) to replace words based on context.
 - **Back Translation:** Translates text into another language and back to the original language using neural machine translation models (generative).
 - **Paraphrasing:** Generates alternative sentence structures with the same meaning using sequence-to-sequence models (generative).

Python Libraries for Image Data Preprocessing and Augmentation

- NLTK (Natural Language Toolkit): One of the oldest NLP libraries. Provides tokenization, stopword removal, stemming and lemmatization, and wordNet-based synonym lookup.
- spaCy: Fast, production-ready NLP library. Provides tokenization, part-of-speech tagging, named entity recognition (NER).
- scikit-learn: Provides bag-of-Words, TF-IDF vectorization.
- TensorFlow / Keras (Text APIs): Provides tokenizers, vocabulary handling embedding layers and sequence padding.
- PyTorch: Provides dataset loaders, tokenization and vocabulary handling.
- Gensim: Specialized for topic modeling and word embeddings. Provides Word2Vec and FastText embeddings.
- EDA (Easy Data Augmentation): provides random deletion, random insertion, random swap, dictionary-based synonym replacement
- Hugging Face Transformers: Library for state-of-the-art NLP deep learning models. Provides BERT, GPT, RoBERTa, T5, etc. Also provides paraphrasing, back translation, and contextual word replacement.

Some Commonly Used Image Datasets

- **IMDB Movie Reviews:** 50,000 movie reviews labeled as positive or negative; commonly used for sentiment analysis.
- **Reuters News Dataset:** News articles / short documents labeled with over 46 topics / classes; used for text classification.
- **20 Newsgroups Dataset:** Collection of 20,000 newsgroup posts across 20 topics.
- **AG News Dataset:** News articles categorized into 4 classes; each class contains 30,000 training samples and 1,900 testing samples.
- **Amazon Product Reviews:** Reviews with ratings across many product categories
- **Wikipedia Text Corpus:** Large collection of encyclopedic text; unsupervised.
- **Common Crawl:** Massive dataset of text scraped from the web.

Data is the new oil!